

Lecture 4: Control Flow and Functions

Beginner Notes | Python for GenAI

1. Conditional Statements

Conditional statements allow a program to make decisions. Python uses **if**, **elif**, and **else** to run different code depending on a condition.

```
age = 20

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

The condition is checked from top to bottom. The first condition that is True is executed.

```
marks = 82

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
else:
    grade = "C"

print(grade)
```

Conditions can use comparison and logical operators such as **==**, **>**, **<**, **and**, and **or**.

2. Loops

Loops repeat a block of code. They are useful when the same operation needs to be performed multiple times.

for loop

A **for** loop is commonly used to go through items in a collection or a sequence of values.

```
languages = ["Python", "Java", "C++"]

for language in languages:
    print(language)
```

A **range()** can generate a sequence of numbers.

```
for number in range(1, 6):
    print(number)
```

while loop

A **while** loop repeats as long as its condition remains True.

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

3. break and continue

break stops the loop completely. **continue** skips the current iteration and moves to the next one.

```
for number in range(1, 6):
    if number == 3:
        break
    print(number)
```

The loop stops when **number** becomes 3.

```
for number in range(1, 6):
    if number == 3:
        continue
    print(number)
```

Here, 3 is skipped, but the loop continues with the next value.

4. Functions

A function is a reusable block of code that performs a specific task. Functions help reduce repetition and make programs easier to organize.

```
def greet():
    print("Hello!")

greet()
greet()
```

A function is defined using **def**. The indented code below the function definition is its function body. The function runs when it is called.

5. Function Arguments

Arguments allow us to send information into a function. The variable written in the function definition is called a parameter.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Rahul")
greet("Priya")
```

A function can accept multiple arguments.

```
def add(a, b):
    print(a + b)

add(10, 20)
```

You can also provide default values for parameters.

```
def greet(name="Student"):
    print(f"Hello, {name}!")

greet()
greet("Rahul")
```

6. Return Values

The **return** statement sends a result back to the place where the function was called. This allows the returned value to be stored or used in another calculation.

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

A function can return a value without printing it. This makes the function more reusable.

7. Recursion

Recursion is when a function calls itself. A recursive function needs a **base case** that stops the repeated calls.

```
def countdown(n):
    if n == 0:
        return
    print(n)
    countdown(n - 1)

countdown(5)
```

Here, **n == 0** is the base case. Without a stopping condition, a recursive function can continue calling itself until Python raises an error.

8. Lambda Functions

A lambda function is a small anonymous function written in a single expression. It is useful for short operations where creating a full function with **def** would be unnecessary.

```
square = lambda x: x * x

print(square(5))
```

A lambda can also take more than one argument.

```
add = lambda a, b: a + b
print(add(10, 20))
```

9. enumerate()

The **enumerate()** function lets you loop through a collection while getting both the position and the value.

```
languages = ["Python", "Java", "C++"]

for index, language in enumerate(languages):
    print(index, language)
```

You can start the index from another number by providing a **start** value.

```
languages = ["Python", "Java", "C++"]

for index, language in enumerate(languages, start=1):
    print(index, language)
```

10. Practical Use Cases

Concept	Practical use case	Example
if / elif / else	Make decisions	Check whether a user is eligible
for loop	Process a collection	Read each product in a shopping cart
while loop	Repeat until a condition changes	Keep asking until valid input is entered
break	Stop a loop early	Stop searching after finding an item
continue	Skip selected items	Ignore invalid records
Function	Reuse a task	Calculate a total or format text
return	Send a result back	Return a calculated score
Recursion	Solve repeated nested problems	Process nested structures
Lambda	Perform a short operation	Quick transformation or sorting rule
enumerate()	Get index and value together	Display numbered items

11. Quick Cheat Sheet

Concept	Remember
if / elif / else	Run different code based on conditions.
for	Repeat code while going through a sequence or collection.
while	Repeat code while a condition is True.
break	Stop the current loop completely.
continue	Skip the current iteration and continue the loop.
def	Define a reusable function.
parameter / argument	A parameter is defined by the function; an argument is the value passed to it.
return	Send a result back from a function.

Concept	Remember
recursion	A function calling itself with a stopping condition.
lambda	Create a small anonymous function in one expression.
enumerate()	Get both the index and value while looping.

Key idea: Control flow decides what code should run and how often. Functions package useful logic into reusable blocks. Together, these concepts form the foundation for writing larger Python programs.